

Response to the Reviewers

Manuscript: *FL-Iroh: NAT-Transparent Federated Learning for Agricultural and Environmental IoT via QUIC P2P Transport and CoAP Service Discovery*

Journal: *Internet of Things* (Elsevier)

Dear Editor, dear Reviewers,

We thank both reviewers for their careful reading and constructive comments. They led us to run a substantial set of new experiments on real hardware and to correct the analysis of our original connectivity results. In summary, the revision:

- adds new experiments: RFC 4787 classification of all new networks, including a mobile-carrier CGNAT (Table 6); forced relay, link outages and network switches on a Raspberry Pi (E8); a packet-loss/latency/bandwidth sweep with 320 federated rounds (E9); public-key admission control on loopback and over the WAN (E10); a complete federation over the WAN with a Raspberry Pi client and per-round CPU, memory and energy (E11); a live head-to-head comparison with Flower over Tailscale on the same hardware (§4.12); and CoAP discovery scaling to 5,000 endpoints with a resource directory (E5, replaced);
- implements and evaluates federation admission control, an authenticated control channel that traverses NAT, transfer retry and endpoint recovery;
- **corrects** the path classification of the original NAT-traversal results. Iroh’s “mixed” state, which we had counted as a direct path, is relay-carried. The corrected figures are reported in E3, and the “0% relay” statements have been removed;
- **corrects** the evaluation of the Prophet-IID configuration, which had been scored on a single zone and is not comparable with the other rows (E6), and the hardware description: the edge device is a Raspberry Pi 3 Model B, not a Raspberry Pi 4B;
- reframes FedGAM as a case study, restricts the privacy and agnosticism claims, and adds a related-work comparison table, 19 recent references (2023–2026), an abbreviation list, an expanded architecture section with a sequence diagram and the round algorithm, and a research roadmap.

Below we answer each comment. Section, table and experiment numbers refer to the revised manuscript. A version with all changes marked is provided as a separate file. All code, per-run results and analysis scripts are in the public repository (branch `revision-r1`).

Reviewer 1

1) My first concern is the generalizability of the NAT-traversal results. Although 96.7% direct-path rate over 150 cold-start attempts is encouraging, all of these experiments ultimately come from a single PC–Raspberry Pi hardware pair and one ISP pairing, and the NAT types were not independently verified using RFC 4787-style mapping/filtering tests. You should repeat the experiment across several routers, mobile operators, ISPs, and geographically separated peers before making broad claims about NAT/CGNAT transparency.

Response. We agree, and addressing this comment led to the most important correction in the revision.

(a) *NAT classification.* We implemented an RFC 4787/RFC 5780 STUN-based classifier (`scripts/nat_classify.py`). It runs on the device itself, queries five STUN servers on different IPs from one local port to classify mapping, and uses CHANGE-REQUEST tests to classify filtering. We

classified every network used in the new experiments: an enterprise office network, a mobile 4G carrier reached through a phone hotspot (two NAT layers), and a residential fixed-line network seen from the host OS and from WSL2 (Table 6). All four show endpoint-independent mapping and address-dependent filtering; the mobile carrier and WSL2 do not preserve ports. The scenarios of the original campaign were not probed at the time, and we now describe them as configured topologies.

(b) *Additional networks and peers.* The new experiments use three further networks: office, mobile CGNAT and residential, in different locations and with different ISPs and one mobile operator. They also add two new peer pairings (Raspberry Pi in the office or on 4G ↔ PC in the office or at home), on top of the original cross-ISP pair 300 km apart.

(c) *Correction of the original results.* While designing the forced-relay experiment (comment 2) we found that with all UDP blocked, so that hole punching is impossible, Iroh reports the path as “mixed” rather than “relay”. In the mixed state the relay carries the traffic while an unvalidated direct candidate is kept. Our original classifier counted “mixed” as direct. We re-analysed the debug logs of all 150 original cold starts, which record whether any direct candidate had been validated. With the strict criterion, 115 of the 120 WAN cold starts were established (95.8%) but only 64 (53.3%) over a validated direct path within 5 s. The port-443 scenario was entirely relay-carried, and the scenario labelled symmetric NAT was half relay-carried (Table 9). We have removed the 98.3%/96.7% “direct” and “0% relay” claims from the abstract, highlights, results and conclusion, and explain the correction in E3. In long-lived sessions, which is the regime of FL, the steady-state path between endpoint-independent NATs is direct: 138/140 transfers on 4G and 60/60 transfers home→office (E8).

(d) *Remaining limits.* We still cover one client device, five probed networks in Spain and the original scenarios. We state this in the Limitations, and multi-operator, multi-country campaigns are the first item of the roadmap. We have toned down all claims of general “NAT/CGNAT transparency” accordingly.

Changes in the manuscript. Abstract; Highlights; §1; §4.1 and Table 6 (networks and RFC 4787 classification); §4.4/E3 rewritten, with Table 9 (strictly classified cold starts) replacing the former transfer-level table; §4.8/E8; §5.1; Limitations; Conclusion.

2) Relay is described as a guaranteed fallback, yet interestingly it was never actually selected in the 150 cold-start experiments. [...] create a network configuration in which UDP hole-punching cannot succeed and then evaluate relay establishment latency, reliability, throughput, and recovery behavior.

Response. Done (new E8). On a Raspberry Pi connected through a mobile CGNAT we blocked all outgoing UDP except DNS with an nftables rule, which makes hole punching impossible. The relay uses HTTPS over TCP and remains available. We alternated blocked and open periods during a 40-minute run with 1 MB transfers every 5 s. Results (Table 14):

- **Reliability.** With UDP blocked, all 190 transfers were delivered over the relay.
- **Fallback latency.** After blocking, the first delivery over the relay came after about 28 s. The in-flight transfer stalled but was not lost.
- **Throughput.** The relay delivered 1.85 Mbps, equal to the direct path on that link (1.88 Mbps), because the 4G uplink was the bottleneck.
- **Recovery.** After unblocking, the path returned to direct within 1.9–3.8 s.

The corrected analysis of comment 1 also shows that the relay was in fact used in the original campaign: all 30 cold starts of the port-443 scenario were relay-carried. Finally, in the complete WAN federation (E11) 56 of the Raspberry Pi’s 58 transfers were relay-carried, because the aggregator

ran behind an additional NAT, and all rounds completed. We have removed the word “guaranteed” and describe the relay as an operational component whose capacity matters.

An unexpected finding is that on the mobile CGNAT the direct path was *less* reliable than the relay. 5–27% of direct transfers were interrupted mid-stream, versus 0 of 190 relay-carried transfers. This motivated a transfer-level retry, now part of the system (§3.3).

Changes in the manuscript. New §4.8 (E8) and Table 14; §3.3; §5.1.

3) Table 8 is essentially an architectural comparison, while the accuracy experiment uses an in-process Flower-compatible harness rather than a real Flower deployment over an actual Tailscale mesh. [...] a live head-to-head comparison on the same hardware would be much more persuasive.

Response. Done. We ran both stacks live on the same machines and network: aggregator on the PC, the Raspberry Pi 3B and two PC clients, the air-quality task, 30 rounds, with server-side evaluation in both. For Flower 1.21 the Raspberry Pi reached the server through Tailscale; for FL-Iroh it registered by public key. Results (new Table 20):

- **Completion and accuracy.** Both completed 30/30 rounds, with similar final accuracy (48.4% Flower, 49.6% FL-Iroh).
- **Round time.** The median round took 8.4 s with Flower over Tailscale and 2.2 s with FL-Iroh (p95 13.8 vs. 3.3 s). We did not profile the source of Flower’s longer rounds, so we report the difference without attributing it.
- **Resources on the Raspberry Pi** (FL client plus `tailscaled` in both runs): CPU time per round 5.5 vs. 6.1 s; memory 440/445 vs. 432/479 MB (median/peak); estimated energy per round 18.0 vs. 10.3 J, mainly because the rounds were shorter.
- **Setup.** Because the Flower server ran behind the host’s NAT, making it reachable over the tailnet required an administrator to add a port-forwarding rule and a firewall exception; FL-Iroh required no network configuration.

The live run also exposed a race in our transfer retry that could deliver a model twice. We fixed it before the final runs (§4.11).

Changes in the manuscript. §4.12 (live comparison) and new Table 20; Table 18.

4) [...] there are currently no measurements of CPU utilization, memory consumption, or energy use on the Raspberry Pi. [...]

Response. Added (E11, Table 17). We implemented a resource monitor that records per-phase CPU time, resident memory, system load and power, and used it on the Raspberry Pi 3B during a complete federation over the WAN. Per round (medians over 29 rounds):

- **Local training:** 1.02 s of wall time and 2.79 s of CPU time.
- **Communication phases:** receiving and sending take 3.1–3.5 s of wall time, but only 0.1–0.2 s of CPU time.
- **Memory:** the Python/PyTorch runtime occupies 255–419 MB, about 40% of the device’s RAM.

Memory, rather than computation, is therefore the binding constraint. This supports our recommendation of a native client runtime (§5.3).

The Raspberry Pi 3B has no power sensor, so energy is *estimated* with a utilization model based on published idle and full-load board power (1.4/3.7 W). We report it explicitly as an estimate: about 2.9 J for training and 4.5–5.2 J for the communication phases, which are dominated by idle waiting. The monitor reads hardware sensors (an INA219/INA226 shunt via Linux `hwmon`, or the PMIC of a Raspberry Pi 5) when they are present, and measured energy is listed in the roadmap.

On removing the system daemon: the live comparison of comment 3 measured the FL client together with `tailscaled` on the same Raspberry Pi. CPU and memory were similar for the two stacks (FL-Iroh used about 10% more CPU time per round, because the QUIC stack runs inside the client process), so removing the daemon does not, by itself, yield a measurable resource saving on this device. We state this explicitly in the paper. The estimated energy per round was lower with FL-Iroh (10.3 vs. 18.0 J), because its rounds were shorter. FL-Iroh’s advantage is therefore operational (no privileged daemon, account or port forwarding) and in round latency, not in CPU or memory.

Changes in the manuscript. New §4.11 (E11) and Table 17; §4.12 and Table 20; §5.5 (Limitations, Energy).

5) [...] most of the FL experiments themselves are executed on a 16-core Xeon HPC system [...] run complete multi-round FL training on multiple Raspberry Pi-class devices [...]

Response. We ran a complete multi-round federation over the WAN (E11). The aggregator ran on a PC at one site (residential network) and the Raspberry Pi 3B at another (office network). Both registered by public key over the NAT-traversing control channel, without port forwarding, VPN or public IP. Two further clients ran on the PC. The Raspberry Pi joined after the first round and took part in every subsequent round: 29 of 30 rounds completed, 27 of them with all three clients, at a median of 7.9 s per round. The 30th round failed because of a race in our transfer retry, which we found in this experiment and fixed: it could deliver a model twice and desynchronize clients. After the fix, the live comparison on the same hardware (comment 3) completed 30 of 30 rounds with all clients.

We had access to only one Raspberry Pi during the revision period, so the federation includes one physical edge device. We state this as a limitation. The statistical experiments (E2, E4, and E6 with five seeds) remain on the server. We now say explicitly that they study learning dynamics, and that all transport behavior is evaluated on real Iroh connections (E1, E3, E6, E8–E11).

During this experiment we also found that the default PyPI PyTorch wheel for 64-bit ARM crashes with an illegal-instruction error on Cortex-A53/A72 boards. We document this deployment pitfall, together with its fix.

Changes in the manuscript. §4.1 (“Transport in each experiment”); new §4.11 (E11); Limitations.

6) Also, expand the related work with ‘ieeexplore.ieee.org/document/11549842’ [...]

Response. We read the paper (Bayraktar and Bakirci, ICIEAM 2026). It is now cited in the new subsection on communication-efficient and resource-aware FL, as an example of compressed, energy- and latency-aware synchronization for intermittent edge–cloud participants. It is also included in the comparison table (Table 2), where we contrast it with FL-Iroh: it reduces what is sent, whereas FL-Iroh addresses whether the aggregator can be reached at all.

Changes in the manuscript. §2.2; Table 2.

7) Current churn experiment models dropout by simply withholding randomly selected clients [...] physically interrupting client connectivity during training, changing its IP/NAT mapping, and then measuring reconnection time and whether the client can safely rejoin subsequent rounds.

Response. Added (E8 and E11). We renamed E4 “robustness to client dropout (simulated)” and state what it does and does not measure. On the Raspberry Pi we then physically took the Wi-Fi interface down for 60 s and 120 s and switched between the mobile 4G network and the office network, which changes the local address and the NAT mapping.

- **60 s outage:** no transfer was lost, and delivery resumed 15.6 s after the link returned.

- **120 s outage:** two transfers timed out during the outage, and delivery resumed immediately afterwards.
- **4G→office switch:** delivery resumed after 4.3 s, over a direct path.
- **Office→4G switch:** the Iroh endpoint did not recover by itself. We report this openly and added an endpoint watchdog that restarts it with the same key, so the node ID is preserved. The watchdog is covered by automated tests, but its field validation remains future work.

Rejoining at the FL level is demonstrated in E11, where the Raspberry Pi joined a running federation and was included from the next round. The aggregator accepts at most one update per selected node ID and round, so a rejoining client cannot double-count.

Changes in the manuscript. §4.5 (E4 scope); new §4.8 (E8); §4.11 (E11); §3.3 (watchdog); Limitations.

8) I would also like to see degraded-network experiments. [...] A controlled packet-loss/latency sweep would be particularly useful for showing when QUIC's behavior begins to affect FL round duration or convergence.

Response. Added (E9, Table 15). We ran the full federated workload (10 clients, 20 rounds) and a transfer benchmark (100 KB, 1 MB, 10 MB) over real Iroh connections, inside an isolated network namespace with `tc-netem`. The conditions were: one-way delay of 25–250 ms, jitter, loss of 1–20%, rate limits from 10 Mbit/s to 512 kbit/s, and two combined rural profiles. All 320 rounds of the 16 completed conditions finished. For 14 KB edge-model updates, impairments lengthen rounds (1.4 s to 3.7 s at 250 ms delay, 8.7 s at 20% loss and 10.7 s at 512 kbit/s) but do not change accuracy or macro-F1. The exception is 20% loss, where late updates reduced participation in two rounds, and the final round aggregated only five clients.

QUIC's behavior becomes visible with payload size. 10 MB transfers grow to 9.5 s at 250 ms delay and to 167 s at 512 kbit/s, and they fail entirely under the harsh rural profile (75 ± 25 ms, 10% loss, 1 Mbit/s). A jitter profile that strongly reorders packets stalled the transfers; we report and exclude it.

Changes in the manuscript. New §4.9 (E9) and Table 15.

9) Scalability experiment for CoAP discovery stops at 100 nodes [...] What happens at 500, 1,000, or several thousand endpoints, especially if multiple capability tags and simultaneous registrations are involved?

Response. E5 has been redone and extended (Table 11). We first found and fixed a flaw in the original measurement: it queried a single server repeatedly and extrapolated byte counts. The new experiment runs one real CoAP server per endpoint, up to 1,000. It also adds an RFC 9176-style resource directory, in which endpoints register once with multiple capability tags and the aggregator answers a single lookup with server-side filtering, up to 5,000 endpoints.

- **Probe discovery** grows linearly: 0.8–1.0 s and 460–580 KB at 1,000 nodes, regardless of how many nodes match.
- **Resource directory** cost depends on the number of matching nodes: a three-tag query over 5,000 endpoints takes 18.9 ms and 19.6 KB.
- **Concurrent registration** of all 5,000 endpoints succeeded without failures (median 220 ms per registration).

Changes in the manuscript. §3.5 (resource directory); §4.6 (E5) and Table 11 replaced.

10) [...] the current implementation does not enforce an allow-list [...] implement and experimentally evaluate this mechanism, including unauthorized-client attempts.

Response. Implemented and evaluated (§3.2, E10).

- **Persistent identity.** Every node now has a persistent Ed25519 identity.
- **Transport-level check.** The aggregator checks the authenticated key of every incoming connection against an allow-list *before* reading any data, and rejects registrations from unknown keys.
- **Endpoint binding.** The NAT-traversing control channel requires the registered endpoint to belong to the authenticated sender.
- **Per-round binding.** Each round accepts at most one update per selected member.

Results (Table 16): on loopback and over the WAN, 0 of 30 unauthorized payloads were accepted. All 30 attempts were rejected and logged, in 10 ms on loopback and 0.33 s over the WAN. The allow-list adds no measurable latency to authorized transfers (11.0 vs. 11.1 ms), and a spoofed registration was rejected.

Changes in the manuscript. §3.2; §4.10 (E10) and Table 16; §5.4; Table 18.

11) Privacy claims should also be phrased carefully. [...] You should avoid broadly describing the overall learning system as privacy-preserving.

Response. We agree and have revised the wording throughout. FL-Iroh is now described as keeping raw data on the device and encrypting updates in transit, not as a privacy-preserving learning system. The threat model (§5.4) adds an explicit “Privacy scope” paragraph. It states that the aggregator receives individual updates in the clear, which may leak information through gradient inversion or membership inference, and that no differential privacy or secure aggregation is applied. It also states that admission control does not protect against poisoned updates from members. Secure aggregation, differential privacy and robust aggregation are now part of the roadmap.

Changes in the manuscript. §1; §5.4; Conclusion; Table 21.

12) Air-quality experiment [...] Report macro-F1, per-class precision/recall, balanced accuracy, and a confusion matrix.

Response. Added. The code now stores all predictions and computes accuracy, balanced accuracy, macro- and weighted-F1, per-class precision/recall/F1 and the confusion matrix. Table 12 reports these metrics for all models and the trivial baselines over five seeds. For example, the majority-class baseline reaches 22.9% accuracy but only 33.3% balanced accuracy and 12.4% macro-F1. The new metrics change the picture in an informative way:

- **Persistence baseline.** It reaches 47.5% accuracy but only 43.1% macro-F1.
- **AirMLP** does not beat persistence on macro-F1 (40.3% geographic, 43.1% IID). It almost never predicts the middle class *Medio* correctly (recall 15–17%).
- **AirLSTM**, added in this revision with a 7-day input window, improves every metric over persistence by about 5–6pp (macro-F1 48.1–49.1%) and nearly doubles the recall of *Medio* (28%). *Medio* remains the hardest class for all models.
- **Prophet**, evaluated over all ten zones, reaches 49.6–49.8% macro-F1, on par with AirLSTM and above persistence. Its higher accuracy (56.7%) comes largely from the majority class.
- **A correction.** These metrics exposed a flaw in our original evaluation: in the IID configuration the Prophet models were scored on the test year of a single zone. There they reach only 35.7–35.8% macro-F1 and almost never predict the class *Alto*. The 57.9% accuracy we reported for Prophet-IID is therefore not comparable with the other configurations. We now state this

explicitly, keep the rows for transparency, and base all comparisons on the evaluation over all zones.

- **FedGAM vs. FedAvg.** The two remain indistinguishable on every metric (49.8% vs. 49.6% macro-F1).

We also add row-normalized confusion matrices (Table 13). Full per-class reports and confusion matrices for every configuration and seed are in the repository.

Changes in the manuscript. §4.7 (E6), Table 12 and new Table 13.

13) FedGAM [...] I wonder whether FedGAM is mature enough to be presented as a substantive contribution in its current form.

Response. We agree. FedGAM is no longer listed as a contribution. It is now presented as a case study of a model-specific aggregation rule that runs over the transport without changes, and the same-model ablation is reported as showing no benefit with eight years of local history. We also give its update rule formally (Eq. 2).

Changes in the manuscript. §1 (contributions); §3.7; §4.7; Limitations.

14) A relevant effort that should also be discussed is ‘10.18280/ts.400524’ by Bakirci [...]

Response. We read the paper (Bakirci, *Traitement du Signal* 40(5), 2023). It presents a swarm-UAV system with onboard object detection and congestion-aware mesh networking among Raspberry Pi/Jetson nodes. It does not address federated learning or Internet-scale connectivity, so we cite it briefly in the related-work subsection on decentralized networking among resource-constrained edge platforms (§2.7).

Changes in the manuscript. §2.7.

15) [...] You should distinguish more carefully between model/aggregation agnosticism and runtime/platform agnosticism.

Response. We agree and added a dedicated subsection (§5.3). The transport is agnostic to the aggregation rule and the model, but not to the client runtime or platform. The prototype serializes with `torch.save` and requires Python/PyTorch on a 64-bit Linux system, which takes about 40% of the Raspberry Pi 3B’s memory (E11) and excludes microcontrollers. We explain what a non-Python client would require: a framework-neutral tensor encoding and a native client speaking the three ALPN protocols, which Iroh’s Rust implementation makes feasible. We claim aggregation and model agnosticism only.

Changes in the manuscript. §1; §3.4; new §5.3; Table 21.

Reviewer 2

1. [...] compare the recent studies in the aspects of problem statements, innovations, methodology, results, and the research gaps [...] add your paper in the last row [...]

Response. Added (Table 2, §2.8). The table compares 14 recent works (2023–2026) on exactly these aspects, with FL-Iroh in the last row. The text identifies three recurring gaps that FL-Iroh addresses: reachability is assumed; connectivity is evaluated in simulation or on one local network; and discovery and membership are not part of the system. It also states the price FL-Iroh pays, namely a heavier client runtime than microcontroller-oriented CoAP systems.

Changes in the manuscript. New §2.8 and Table 2.

2. You use the older sources in your topic. It is important to add the most recent works [...]

Response. We added 19 references from 2023–2026, organized in new or extended related-work subsections. They cover FL for agriculture and environmental monitoring; communication-efficient and resource-aware FL on edge hardware; transport comparisons for distributed learning under disruption; CoAP-based FL on constrained devices; and decentralized FL platforms. We also added the relevant standards (RFC 4787 and RFC 9176). All references were verified against their DOIs.

Changes in the manuscript. §2; Table 2; References.

3. Add the abbreviations to the first of your paper.

Response. Added as Table 1 (“Abbreviations”) at the beginning of the paper.

4. the explications of architecture is not sufficient and I think it needs more details.

Response. Section 3 has been substantially expanded. It now includes:

- a table of software components and their responsibilities;
- a subsection on node identity and admission control;
- a detailed description of Iroh’s path selection, including how we distinguish direct, mixed and relay paths, and of the transfer retry and endpoint watchdog;
- the wire format and acknowledgement semantics;
- the CoAP control plane and the resource directory, with an example lookup;
- a message-sequence diagram of one round (Fig. 4) and the aggregator’s round algorithm (Algorithm 1, Table 4);
- the formal FedGAM update rule (Eq. 2).

Changes in the manuscript. §3 (Tables 3–4, Fig. 4, Eq. 2).

5. You can add a research road map to the conclusion section [...]

Response. Added. The conclusion now ends with a research roadmap (Table 21) organized by horizon. The short-term engineering items cover measurement breadth, operational baselines and measured energy. The medium-term systems items cover lightweight runtimes, communication efficiency, and asynchrony and intermittency. The long-term research items cover trustworthy FL, decentralized topologies and domain models. Each item lists concrete opportunities for other researchers.

Changes in the manuscript. §6 and Table 21.

We hope that the revised manuscript addresses all comments satisfactorily, and we thank the reviewers again for helping us improve the paper.

Sincerely,
The authors